Untitled

DOC APP

1) Operation sur les liste simplemnt chaine :

déclaration d une liste :

```
struct wagon{
  int num;
  int type;
  int etat;
};

typedef struct{
  struct wagon w;
  struct lsc *suiv;
}lsc;
```

Déclaration de la liste

```
struct Cellule
{
int valeur ;
struct Cellule * suivant ;
};
typedef struct Cellule * liste;
int main{
liste l=NULL; // déclaration et initialisation de la liste
}
```

Liste vide

```
int liste_vide(liste l)
{
return (l==NULL);
}
```

1) **Fonction ajouter tete :**

```
  lsc* ajouter_loc(lsc *l,int loc)
{
  lsc *nouv=NULL;
  nouv=(lsc*)malloc(sizeof(lsc));
```

```
    if(nouv==NULL){
        printf("Stop\n");
    }else{
    nouv->w.num =loc;
    nouv ->suiv=l;
    l=nouv;
    }
    return (l);
}
```

	liste ajouter_deb(liste l, int val) {
Déclaration	struct Cellule* nouv;

Allocation dynamique Et remplissage d'une nouvelle cellule	nouv = (struct Cellule*) malloc(sizeof(struct Cellule));
	nouv->valeur=val;
	nouv->suivant=NULL;

Le suivant de la nouvelle cellule est l'ancienne tête	nouv->suivant=l;
La nouvelle tête est l'adresse de la nouvelle cellule	l=nouv;
l'adresse de la première cellule a changé -->elle doit être retournée	return(l); }

2) Fonction ajouter quene :

```
lsc* ajouter_quene(lsc *l)
{
    lsc *nouv = (lsc*)malloc(sizeof(lsc));
    if (nouv == NULL) {
        printf("Stop: Échec de l'allocation mémoire\n");
        return l;
    }
    printf("Saisir le numéro du wagon: ");
```

```
    scanf("%d", &(nouv->w.num));
    do {
        printf("Saisir l'état (1 pour en panne, 2 pour OK): ");
        scanf("%d", &(nouv->w.etat));
    } while (nouv->w.etat != 1 && nouv->w.etat != 2);
    do {
        printf("Saisir le type (1 pour locomotive, 2 pour wagon): ");
        scanf("%d", &(nouv->w.type));
    } while (nouv->w.type != 1 && nouv->w.type != 2);
    nouv->suiv = NULL;
    if (l == NULL) {
        return nouv;
    }
    lsc *parc = l;
    while (parc->suiv != NULL) {
        parc = parc->suiv;
    }
    parc->suiv = nouv;
    return l;
}
```

	liste ajouter_Fin(liste l, int val) {
Déclaration	struct Cellule * nouv, *parc;

Création nouvelle cellule	nouv = (struct Cellule*) malloc(sizeof(struct Cellule));
	nouv->valeur=val;
	nouv->suivant=NULL;

Cas tête ==Null	if(l==NULL)
	{
	l=nouv;
	}

Cas tête !=Null	else
	{
	parc=l;
	while(parc->suivant!=NULL)
	parc=parc->suivant;
	
	parc->suivant = nouv;
	}

Fin	return(l); }

3) Fonction chercher dans une liste :

```
int chercher_wagon(lsc *l, int num)
{
   lsc *p = l;
   while (p != NULL) {
      if (p->w.num == num) {
         return 1; // le num est dans la liste
      }
      p = p->suiv;
   }
   return 0;  // le num nest pas dans la liste
}
```

4) Fonction Afficher liste :

```
void affiche(lsc *l)
{
    lsc *p=l;
    if(p==NULL){
        printf("la liste est vide");
    }else{
        while(p!=NULL){
            printf("Num wagon = %d",p->w.num);
            printf("type wagon = %d",p->w.type);
            printf("etat wagon = %d",p->w.etat);
            p=p->suiv;
        }
    }
}
```

5)Supprimer une wagon dans une liste liste

```
lsc* supprimer_wagon(lsc *l, int num) {
    lsc *courant = l;
    lsc *precedent = NULL;
    if (courant != NULL && courant->w.num == num) {
        lsc *suivant = courant->suiv;
        free(courant);
        return suivant;
    }
    while (courant != NULL && courant->w.num != num) {
        precedent = courant;
        courant = courant->suiv;
    }
    if (courant == NULL) {
        printf("Le wagon avec le numéro %d n'est pas dans la liste\n", num);
        return l;
    }
    precedent->suiv = courant->suiv;
    free(courant);
    printf("Le wagon avec le numéro %d a été supprimé de la liste\n", num);
```

```
    return l;
}
```

6) Fonction libérer liste

```
void detruire_liste(lsc *l)
{
    lsc *courant = l;
    lsc *suivant;
    while (courant != NULL) {
        suivant = courant->suiv;
        free(courant); // Libérer la mémoire du maillon courant
        courant = suivant; // Passer au maillon suivant
    }
    printf("La liste a été détruite avec succès\n");
}
// À la fin de votre programme, lorsque vous avez terminé d'utiliser la liste
detruire_liste(train);
```

7)ajouter au milieu de la liste

	liste ajoutmilieu (liste l , int val, int critere) {
Déclaration	struct Cellule* nouv=NULL, *precedent=NULL,*courant=l;

Recherche de la cellule vérifiant le critère et de la cellule qui la précède	while (courant!=NULL && courant->valeur !=critere)
	{ precedent=courant;
	courant=courant->suivant;
	}

Pas d'insertion	if(courant==NULL)//liste vide ou critère non vérifié dans toutes les cellules.
	printf(«critère non vérifié »);
	else //critère vérifiée

Allocation et remplissage d'une nouvelle cellule	{
	nouv=(struct Cellule*)malloc(sizeof(struct Cellule));
	nouv->valeur=val;

Changement de la tête	nouv->suivant=courant;
	If(courant==l)
	l=nouv;

Changement du chainage	else
	precedent->suivant=nouv;
	}

Ne pas perdre la nouvelle tête	return l; }

8) Suppression au début de la liste

	liste supprimerDebut(liste l) {
Déclaration	struct Cellule* tmp;

Cas liste Non vide	if(l!=NULL)
	{
	tmp =l;// pointer sur le premier élément de la liste
	l=l->suivant; //pointer la tête sur le deuxième élément de la liste
	
	free(tmp); // détruire la cellule
	}

Fin	return(l); }

9)Suppression a la fin de la liste

	liste supprimer_Fin(liste l) {
Déclaration	struct cellule* tmp, courant;

Cas d'une liste qui contient une seule cellule	if (l!=NULL) // si la liste n'est pas vide
	{
	if(l->suivant==NULL) // s'il n'y a qu'une seule cellule
	{
	free(l);
	l=NULL; }

Cas liste qui Contient au moins deux cellules	else.// s'il y a au moins deux cellules
	{tmp=l;
	courant=l;
	// à partir du premier,
	while(tmp->suivant!=NULL) // regarder toujours le suivant du suivant pour accéder à l'avant dernier
	{
	courant =tmp;
	tmp=tmp->suivant;
	}
	courant->suivant=NULL;
	free(tmp);
	}
	}

Fin	return l; }

2) FICHIERS TXT /BIN:

Operation sur les fichiers text :

ecrire dans *un fichier* :

```
    Fonction ecrire(char name[]){
    FILE *p=NULL;
   p= fopen(name,"w");
 if (p==NULL){
        printf("Stop\n")
} else{
       fprintf(p,"Wassim age %d",age);
}
fclose(p);
}
}
```

Lire dans *un fichier* :

```
 FILE *p=NULL;
   p= fopen(name,"w");
 if (p==NULL){
        printf("Stop\n")
} else{
while(feof(p)){
      fscanf(p,"Wassim age %d",age);
}
fclose(p);
}
}
```

chercher un elemnts dans *un fichier* :

```
FILE *p=NULL;
```

```
   p= fopen(name,"w");
  if (p==NULL){
        printf("Stop\n")
} else{
while(feof(p)){
      fscanf(p,"Wassim age %d",age);
      if(age==nombre){
         return 1;
      }else{
          return 0;
  }
}
fclose(p);
}
}
```

supprimer un fichier :

```
void supprimer(char name[])
{ struct employe e;
FILE *p=NULL;
p = fopen(name, "r");
if(p == NULL){
printf("Error opening file for reading\n");
} else {
FILE *supp = fopen("supp.txt", "w");
if(supp == NULL){
printf("Error opening supplementary file\n");
} else {
while(fscanf(p, "%s %s %d", e.name, e.per, &e.nbf) == 3){
if(e.nbf != 0){
fprintf(supp, "%s %s %d\n", e.name, e.per, e.nbf);
}
}
fclose(p);
fclose(supp);
```

```
remove(name);
rename("supp.txt", name);
}
}
}
```

Operation sur les fichiers binaire :

Écrire dans un fichier binaire :

```
void ecrire_binaire(char name[], int age) {
   FILE *p = fopen(name, "wb"); // Ouverture en mode binaire
   if (p == NULL) {
      printf("Erreur: Impossible d'ouvrir le fichier\n");
   } else {
      fwrite(&age, sizeof(int), 1, p); // Écriture de l'âge dans le fichier binaire
      fclose(p);
   }
}
```

Lire à partir d'un fichier binaire :

```
void lire_binaire(char name[]) {
   int age;
   FILE *p = fopen(name, "rb"); // Ouverture en mode binaire
   if (p == NULL) {
      printf("Erreur: Impossible d'ouvrir le fichier\n");
   } else {
      fread(&age, sizeof(int), 1, p); // Lecture de l'âge depuis le fichier binaire
      printf("Age: %d\n", age);
      fclose(p);
   }
}
```

Chercher un élément dans un fichier binaire :

```
int chercher_binaire(char name[], int nombre) {
    int age;
    FILE *p = fopen(name, "rb"); // Ouverture en mode binaire
    if (p == NULL) {
        printf("Erreur: Impossible d'ouvrir le fichier\n");
        return 0; // Si le fichier n'existe pas, retourne 0
    } else {
        while (fread(&age, sizeof(int), 1, p) == 1) { // Lecture de chaque âge dans le fichier binaire
            if (age == nombre) {
                fclose(p);
                return 1; // Retourne 1 si l'âge est trouvé
            }
        }
        fclose(p);
        return 0; // Retourne 0 si l'âge n'est pas trouvé
    }
}
```

Supprimer

```
void supprimer(char name[]) {
struct employe e;
FILE *p = NULL;
p = fopen(name, "rb");
if (p == NULL) {
printf("Erreur lors de l'ouverture du fichier en lecture\n");
} else {
FILE *supp = fopen("supp.txt", "wb");
if(supp == NULL) {
printf("Erreur lors de l'ouverture du fichier supplémentaire en écriture\n");
} else {
while (fread(&e, sizeof(struct employe), 1, p) == 1) {
if (e.nbf != 0) {
fwrite(&e, sizeof(struct employe), 1, supp);
}
```

```
}
fclose(p);
fclose(supp);
remove(name);
rename("supp.txt", name); }
}
}
```